

<https://www.di.ens.fr/~nitulesc/files/Survey-SNARKs.pdf> <https://z.cash/learn/>
-> Many sub-sources from this one <https://eprint.iacr.org/2014/349.pdf>

- **zkSNARKs (zero-knowledge succinct non-interactive arguments of knowledge)**

- basically a cryptography thing where someone proves they know something **without revealing the thing itself**
- the name is long but each word means something:
 - * **zero-knowledge** → the verifier learns *nothing except that the statement is true*
 - * **succinct** → the proof is very small and fast to check
 - * **non-interactive** → the prover sends *one proof*, not a back-and-forth conversation
 - * **argument of knowledge** → the prover can only produce the proof if they actually know the secret
- usually used when you want **privacy + verifiability**
- works by converting a computation into an **arithmetic circuit** and then proving the circuit output is correct without revealing the inputs
- there's a trusted setup phase in many systems where some public parameters are generated
- math inside uses:
 - * elliptic curve cryptography
 - * pairings on elliptic curves
 - * polynomial commitments
- typical flow (very simplified):
 - * define a statement like: *"I know a secret value that satisfies these conditions"*
 - * convert the computation into a constraint system
 - * prover generates proof using secret inputs
 - * verifier checks proof quickly using public parameters
- key idea:
 - * **proof size and verification time stay tiny even if the original computation is huge**

- **why zkSNARKs exist (the problem they solve)**

- normal proofs reveal data
- example: proving you paid someone normally means revealing:
 - * your address

- * recipient address
 - * transaction amount
 - sometimes you want to prove **validity of something without revealing private info**
 - zero-knowledge proofs let you say:
 - * “this transaction is valid”
 - * without showing **who sent it or how much**
-

- **ZCash (a cryptocurrency focused on privacy)**

- created in **2016** as a fork of the Bitcoin codebase but with added privacy features
 - goal: make **financial transactions private while still being publicly verifiable**
 - main difference from Bitcoin:
 - * **Bitcoin transactions are transparent**
 - * **ZCash can hide transaction details**
 - the system uses **zkSNARKs to prove transactions are valid without revealing information**
-

- **what normal Bitcoin transactions reveal**

- sender address
 - receiver address
 - transaction amount
 - full transaction history
 - anyone can trace money flow on the blockchain
-

- **what ZCash tries to hide**

- sender
 - receiver
 - amount
 - still keeps proof that:
 - * coins exist
 - * coins weren't double-spent
 - * transaction balances correctly
-

- **ZCash address types**

- **transparent addresses (t-addresses)**
 - * work basically like Bitcoin
 - * transactions visible on blockchain
 - **shielded addresses (z-addresses)**
 - * use zkSNARKs
 - * transaction details are hidden
-

- **how zkSNARKs are used in ZCash transactions**

- instead of revealing transaction info, the sender generates a **zk-SNARK proof** that shows:
 - * they own the coins they're spending
 - * the coins haven't been spent before
 - * total inputs = total outputs
 - the proof is posted to the blockchain
 - validators check the proof without seeing the underlying data
-

- **rough simplified flow of a ZCash shielded transaction**

- user wants to send coins privately
 - wallet constructs a proof that:
 - * user has a valid note (like a hidden coin record)
 - * the note hasn't been spent
 - zkSNARK proof gets attached to the transaction
 - blockchain nodes verify proof
 - if proof checks out → transaction accepted
 - but nodes **never see the secret values**
-

- **why zkSNARKs are good for blockchains**

- proof sizes are **very small** (a few hundred bytes)
 - verification is **fast**
 - perfect for blockchains where every node must verify transactions
-

- **the trusted setup issue**

- early zkSNARK systems require generating secret parameters
 - if someone kept the toxic waste from setup they could:
 - * forge coins
 - ZCash did a **multi-party computation ceremony** to generate parameters and then destroy secrets
 - basically lots of people participated so that no single person controlled the setup
-

- **important concept in ZCash: “notes”**

- instead of visible coins like Bitcoin outputs
 - ZCash uses **notes** which are encrypted coin records
 - when spending a note:
 - * you prove ownership via zkSNARK
 - * you reveal a **nullifier** (prevents double spending)
-

- **nullifiers**

- special value derived from the note secret
 - published when spending
 - if the same nullifier appears twice → network rejects transaction
 - prevents double spending while still hiding the note itself
-

- **downsides / criticisms**

- zkSNARK math is **very complicated**
 - trusted setup requirement makes some people uncomfortable
 - shielded transactions used to require lots of RAM and computation
 - fewer people use shielded mode than expected
-

- **later improvements**

- ZCash later introduced **Sapling upgrade**
 - major improvements:
 - * faster proofs
 - * less memory
 - * mobile wallets possible
-

- **big picture**

- zkSNARKs = tool that allows **private yet verifiable computation**
- ZCash = real world application showing how zero-knowledge proofs can give **financial privacy on a public blockchain**
- kind of a big deal because blockchains normally trade privacy for transparency

- **super short summary**

- zkSNARK → prove something is true without revealing why
- ZCash → cryptocurrency using zkSNARKs to hide transaction details but still verify they're valid